

A Contrastive Language-Image Pre-training (CLIP) Framework for Semantic Image Retrieval

Davide Catucci (2153384) and Sara Berxhaj (2173374)

Sapienza University Of Rome

June 13, 2026

Abstract

Traditional image retrieval systems on mobile devices predominantly rely on coarse-grained categorical labeling, limiting user queries to predefined classes. We address this with a semantic retrieval framework based on the contrastive vision-language paradigm of CLIP [1]. Rather than reusing CLIP’s weights, we reproduce its alignment under an academic compute budget: a frozen ViT-B/16 image encoder (ImageNet-21k) is paired with a trainable BERT text encoder via Locked-image Tuning. Our approach enables fine-grained, natural language search capabilities delivering consistently accurate results within image repositories of size up to few thousands.

1 Introduction

Current image retrieval systems on personal devices primarily rely on categorical labeling. While functional, this approach is often insufficient for precise user queries due to two inherent limitations.

First, traditional classification suffers from a lack of mutual exclusivity; since a single image frequently contains multiple semantic elements, a class-based search often returns a high volume of "noise" or irrelevant results.

Second, and more critically, discrete labels fail to capture fine-grained visual nuances. For instance, a user seeking a specific variety of pizza such as a "Neapolitan pizza with charred edges" is restricted by the broad "pizza" category. In such cases, the system retrieves a massive, undifferentiated dataset, forcing the user to manually filter through hundreds of images, necessitating a move toward natural language-based retrieval frameworks.

	query_text	ground_truth_image	top_1_prediction	top_2_prediction	top_3_prediction	top_4_prediction	top_5_prediction
1	A cake sits on a plate with a knife behind it.		0.72			0.72	
2	Group of sailors in command center with one talking on mobile phone.		0.81				
3	People are looking at computer and one man has a camera.		0.71				
4	A stop sign outside of a brick building.		0.78				
5	A person that is making something out of wood.		0.66				
6	A kitchen filled with lots of counter space.		0.51				
7	two cows stand near each other and look at the camera		0.65	0.65			

Figure 1: Top-5 image retrieval results based on 7 different prompts.

To optimize the CLIP framework, we introduced two distinct modifications to the contrastive loss function:

- **Directional Weighting (Section 2.2):** We transition from a standard symmetric loss to an asymmetric, directionally weighted objective. This formulation specifically prioritizes minimizing the text-to-image contrastive loss, thereby optimizing the network for our exact inference pipeline.
- **Computational Approximation (Section 2.3):** We modify the contrastive formulation using an approximation designed to mitigate computational overhead and accelerate training. To validate the viability of this optimization, a secondary model was evaluated under an abbreviated training regime to gauge its convergence properties and retrieval performance.

2 Model Architecture

The core of our framework is built upon the Contrastive Language-Image Pre-training (CLIP) architecture. Unlike traditional supervised models that rely on discrete class labels, CLIP utilizes a dual-encoder strategy to learn visual representations directly from natural language supervision, enabling a more flexible semantic understanding of image content.

2.1 The CLIP Framework

The architecture consists of two primary components: a Text Encoder (typically a Transformer-based model) and an Image Encoder (such as a Vision Transformer, ViT). The output features of the two encoders are subsequently projected via learned linear transformations into a shared d -dimensional joint multimodal latent space, where their cosine similarity can be computed.

During the training phase, the model is presented with a batch of N (image, text) pairs. The encoders generate embeddings for every possible combination

within the batch, resulting in an $N \times N$ similarity matrix. The model is optimized using a symmetric cross-entropy loss that maximizes the cosine similarity between the N correct pairings—the diagonal of the matrix—while minimizing the similarity for the $N^2 - N$ incorrect pairings.

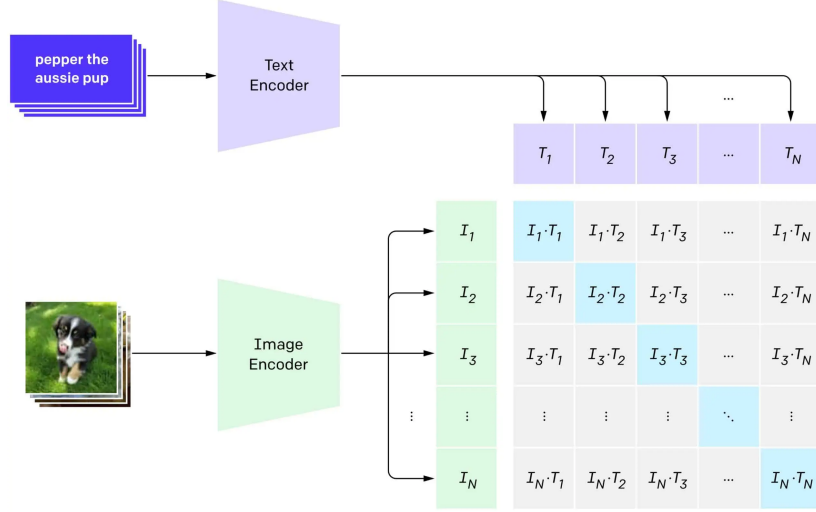


Figure 2: The CLIP Architecture: Independent text and image encoders project data into a shared embedding space, optimized via contrastive alignment.

Mathematically, given an image embedding $\mathcal{I}$ and a text embedding $\mathcal{T}$, the similarity score is calculated as the cosine similarity:

$$\text{sim}(\mathcal{I}, \mathcal{T}) = \frac{\mathcal{I} \cdot \mathcal{T}}{\|\mathcal{I}\| \|\mathcal{T}\|} \quad (1)$$

The training logic, as described in Algorithm 1, focuses on maximizing the cosine similarity of the N paired embeddings while minimizing the $N^2 - N$ incorrect pairings.

Algorithm 1 CLIP Training Logic (Symmetric Contrastive Loss)

```
1: Input: Batch of  $N$  (image, text) pairs  $(I, T)$ 
2:  $I_f \leftarrow \text{ImageEncoder}(I)$   $\triangleright$  Extract image features  $[N, d_i]$ 
3:  $T_f \leftarrow \text{TextEncoder}(T)$   $\triangleright$  Extract text features  $[N, d_t]$ 
4:  $I_e \leftarrow \text{Normalize}(I_f \cdot W_i)$   $\triangleright$  Project and L2 normalize  $[N, d]$ 
5:  $T_e \leftarrow \text{Normalize}(T_f \cdot W_t)$   $\triangleright$  Project and L2 normalize  $[N, d]$ 
6:  $\text{logits} \leftarrow (I_e \cdot T_e^T) \cdot e^\tau$   $\triangleright$  Compute similarity matrix  $[N, N]$ 
7:  $\text{labels} \leftarrow \text{arange}(N)$   $\triangleright$  Targets are the diagonal indices
8:  $\mathcal{L}_i \leftarrow \text{CrossEntropy}(\text{logits}, \text{labels}, \text{axis}=0)$   $\triangleright$  Image-to-text loss
9:  $\mathcal{L}_t \leftarrow \text{CrossEntropy}(\text{logits}, \text{labels}, \text{axis}=1)$   $\triangleright$  Text-to-image loss
10: return  $\frac{\mathcal{L}_i + \mathcal{L}_t}{2}$   $\triangleright$  Symmetric loss combination
```

2.2 Directional Weighting

In the standard CLIP implementation, the model is optimized using a symmetric cross-entropy loss, which is the simple average of the image-to-text loss ($\mathcal{L}_i$) and the text-to-image loss ($\mathcal{L}_t$):

$$\mathcal{L}_{\text{symmetric}} = \frac{\mathcal{L}_i + \mathcal{L}_t}{2} \quad (2)$$

However, to better align the model with our specific goal of natural language-based image retrieval, We introduced a weighted loss function. This allows us to prioritize the text-to-image alignment, which is the only direction of inference in our system. The modified loss function is defined as:

$$\mathcal{L}_{\text{asymmetric}} = \alpha \mathcal{L}_t + (1 - \alpha) \mathcal{L}_i \quad (3)$$

Where $\alpha \in [0, 1]$ is a hyperparameter that controls the directional bias.

By setting $\alpha > 0.5$ (we used 0.8 in our implementation), we place a higher weight on the text-to-image component. This ensures that the shared latent space is structured to be more discriminative toward the nuances of textual queries, thereby improving the precision of the retrieval results in the presence of fine-grained linguistic descriptors.

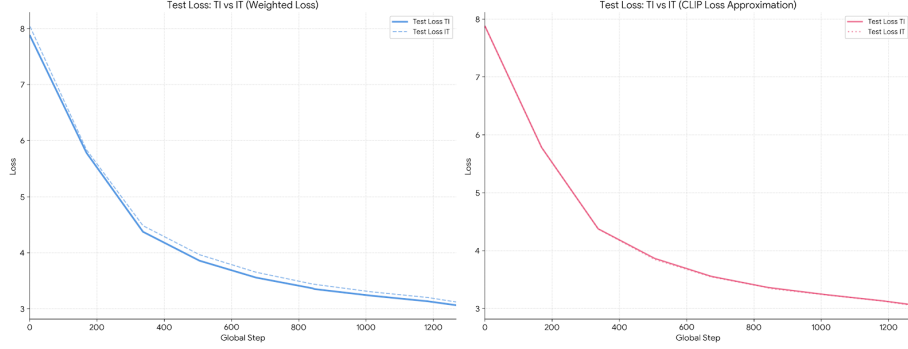


Figure 3: asymmetric vs. symmetric test loss comparison: (Left) TI task prioritization; (Right) Balanced convergence under the standard CLIP objective.

2.3 Customized Objective Function

Our goal was to find an alternative formulation of the cross-entropy loss that would be faster to compute while preserving the essential training signal. This section documents the development process, including the intermediate attempts that did not work as intended, since each step informed the final design.

Starting Point: Standard Cross-Entropy

The cross-entropy loss used by CLIP over a batch of N similarity logits $\mathbf{z} \in \mathbb{R}^N$, where z_j is the logit corresponding to the correct (positive) pair, is:

$$L_{CE}(\mathbf{z}, j) = -z_j + \log \left(\sum_{i=1}^N e^{z_i} \right) \quad (4)$$

The term $\log(\sum_i e^{z_i})$ is the log-sum-exp, acting as a soft maximum over all logits. Our idea was to replace this term with a simpler approximation while maintaining the qualitative behavior of the loss.

First Attempt: Direct Maximum

A natural first simplification is to note that $\log(\sum_i e^{z_i}) \approx \max_i(z_i)$ for logits of comparable magnitude, so the dominant contribution comes from the largest element. This led us to the approximation:

$$L_1(\mathbf{z}, j) = -z_j + \max_i(z_i) \quad (5)$$

The problem with this formulation is that the max operator is not differentiable: its gradient is 1 at the argmax index and 0 everywhere else. In practice this

means only one logit receives a gradient signal at each step, all other elements of $\mathbf{z}$ are ignored by the optimizer.

Second Attempt: Top- k Averaging

To address the extreme sparsity of the gradient in L_1 , we generalized the maximum to the average of the k largest elements of $\mathbf{z}$:

$$L_2(\mathbf{z}, j) = -z_j + \frac{1}{k} \sum_{i \in \text{TopK}(\mathbf{z})} z_i \quad (6)$$

This spreads the gradient over k elements (each receiving $1/k$) rather than concentrating it on one, which is a clear improvement over L_1 . However, the top- k selection is still not differentiable with respect to which indices are selected: the remaining $N - k$ elements still receive zero gradient. In our experiments, gradient instability persisted, especially in early training when logits are not yet well separated.

Final Approximation: ℓ_2 Norm

Both previous attempts share the same root cause: they assign zero gradient to most elements of $\mathbf{z}$, making the optimization landscape poorly conditioned. This motivated us to look for a surrogate that involves all components of $\mathbf{z}$ in a smooth way. We arrived at the ℓ_2 norm of the logit vector:

$$L_3(\mathbf{z}, j) = -z_j + \|\mathbf{z}\|_2 \quad (7)$$

where $\|\mathbf{z}\|_2 = \sqrt{\sum_i z_i^2}$.

The key properties of this approximation are:

- **Dense and bounded gradients:** The partial derivatives are:

$$\frac{\partial L_3}{\partial z_j} = -1 + \frac{z_j}{\|\mathbf{z}\|_2}, \quad \frac{\partial L_3}{\partial z_i} = \frac{z_i}{\|\mathbf{z}\|_2} \quad (i \neq j) \quad (8)$$

Every element of $\mathbf{z}$ contributes to the gradient at every step, and since $z_i/\|\mathbf{z}\|_2 \in [-1, 1]$, the gradients are always bounded and never explode. This directly resolves the sparsity and instability of L_1 and L_2 .

- **Sensible training signal:** The loss is minimized when $z_j \rightarrow \|\mathbf{z}\|_2$, which geometrically means the logit vector is dominated by its j -th component the correct match. This is equivalent to pushing the model toward configurations where the positive pair has much higher similarity than all negative pairs.

The progression from L_1 to L_2 to L_3 can thus be read as a steady improvement along two axes: gradient density (1 element $\rightarrow k$ elements $\rightarrow$ all N elements) and differentiability (nowhere $\rightarrow$ almost everywhere $\rightarrow$ everywhere except $\mathbf{z} = \mathbf{0}$). While L_3 is not a tight approximation of the log-sum-exp, it preserves the essential contrastive structure and proved to be the only formulation among the three that trained stably and is faster.

This formulation was used to train a second version of our model, which we compare against the standard asymmetric cross-entropy baseline in the results.

3 Experimental Setup

This section details the training configuration, hyperparameter selection, and the comparative analysis of various tuning strategies.

3.1 Training data and Batching

For the training phase, we utilized the MS-COCO dataset. This corpus was selected primarily for its high-quality, human-annotated labels and its established benchmark status in multimodal learning. Crucially, MS-COCO provides a favorable balance between semantic complexity and scale, aligning with our computational feasibility and hardware constraints.

The dataset comprises approximately 650,000 image-text pairs, partitioned into a training set of 600,000 samples, with the remaining 50,000 equally distributed between validation and testing sets. A distinct advantage of the MS-COCO architecture is the provision of five unique descriptions per image. We leveraged this multi-captioning structure to enhance the model’s semantic robustness, allowing the network to learn that a single image can be represented through diverse linguistic interpretations.

The model was trained with a batch size of 256, which represented the upper limit of our hardware’s VRAM capacity.

3.2 Hardware

We trained the model on a single NVIDIA A100 24GB SXM4 GPU. Using the hyperparameters described throughout the paper, each training step took about 5 seconds. We trained the model for a total of 1,286 steps or 3 hours, almost 5 epochs.

We utilized a gradient accumulation factor of 8, resulting in elaborating 2048 pairs before updating the model parameters at each step.

While the initial experimental protocol was scheduled for 8 epochs (approximately 2,208 steps), we implemented an early stopping criterion at 1,286 steps. This decision was informed by an analysis of the positive similarity statistics, which exhibited a distinct plateau during the test phase (See Figure 4).

The stagnation of these similarity metrics indicated that the model had reached a point of diminished returns in optimizing the alignment between image and text embeddings. Consequently, further training was terminated to prevent overfitting and to preserve computational resources.

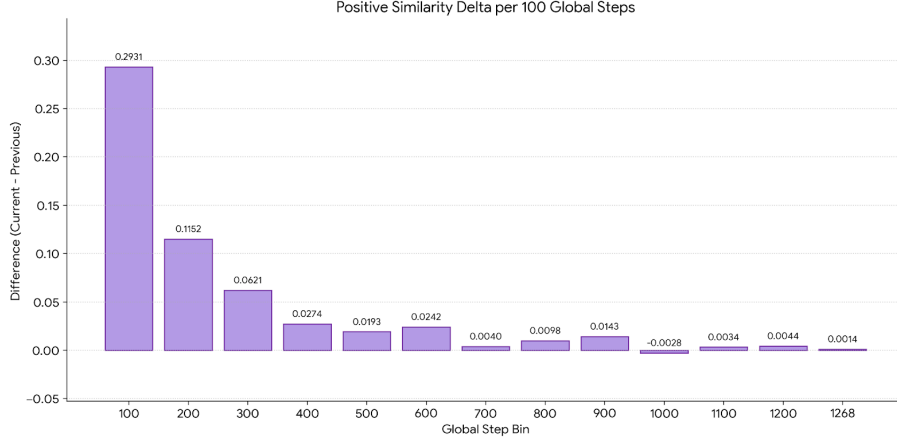


Figure 4: A convergence plateau is presented from step 1000 until step 1268 where we decided to stop the training.

Given the batch size of 256, memory utilization reached 21GB of the available 24GB VRAM. While further memory optimization techniques were considered to marginally increase the batch size, we prioritized training velocity and step-wise throughput.

3.3 Optimizer and Learning Rate Scheduler

The model was optimized using the Adam optimizer with hyperparameters $\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$ and starting learning rate 10^{-4} . We implemented a dynamic learning rate schedule to stabilize early training and ensure long-term convergence. Specifically, the learning rate l_{rate} was adjusted as a function of the step number, according to the following formula:

$$l_{rate} = \begin{cases} l_{max} \frac{t}{T_{warmup}} & t < T_{warmup} \\ 0.5 \cdot l_{max} \left(1 + \cos \left(\pi \frac{t - T_{warmup}}{T_{total} - T_{warmup}} \right) \right) & T_{warmup} \leq t \leq T_{total} \end{cases} \quad (9)$$

This scheduling strategy entails a linear increase in the learning rate for the initial $warmup_steps = total_training_steps * 0.1$. Following this warmup phase, the learning rate decays according to a cosine function. This approach allows the model to explore the loss landscape broadly during the initial stages before refining the weights as training progresses.

3.4 Pre-trained Foundation Models

To capitalize on robust visual and linguistic representations, our architecture leverages pre-trained foundation models as the backbone for feature extraction. For the visual modality, we utilize the Vision Transformer (ViT-B/16) [1], pre-trained on the ImageNet-21k dataset at a resolution of 224×224 pixels. For the textual modality, we employ the BERT-base-uncased model [2].

Given the prohibitive computational cost of fine-tuning the entire architecture, we adopted a Locked Image-Tuning (LiT) [3] approach. By freezing the ViT encoder which accounts for the vast majority of the total parameter count, we significantly reduced the memory footprint and training time.

By "locking" these pre-trained parameters, we restricted the optimization process exclusively to the projection heads and the BERT encoder. This approach significantly reduced the number of trainable parameters, accelerating convergence within our hardware constrained environment.

4 Results

4.1 Main model using asymmetric cross-entropy loss

During the training phase, the model achieved a training loss of 0.83, however a significant divergence was observed when evaluating the model against a large-scale test gallery. Specifically, utilizing a test batch size (gallery size) of 3,072k resulted in a test loss of 2.69, with the Top-5 accuracy stabilizing at 64%.

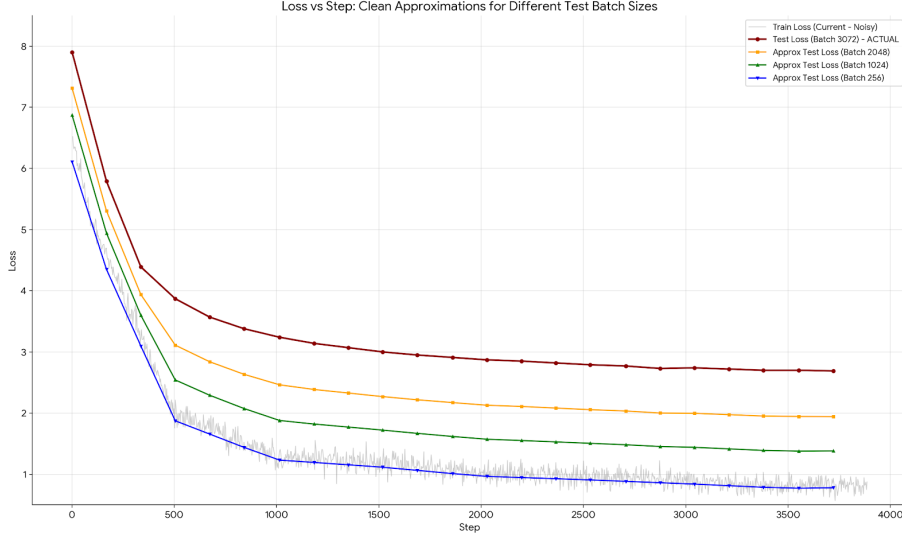


Figure 5: This graph show how different batch size in testing change significantly the test loss

This performance delta highlights the impact of search space cardinality on retrieval precision. While smaller batch sizes (e.g., $N=256$, matching the training distribution) yielded near-saturated accuracy approaching 100% these metrics fail to reflect real-world deployment scenarios. To assess the "true" discriminative power of the learned embeddings, we deliberately scaled the test batch size to 3,072k. This stress test simulates the high-entropy environment of massive data search, providing a more robust measure of the model's ability to distinguish between fine-grained semantic features amidst millions of distractors.

The observed discrepancy between Top-1 and Top-5 accuracy (an increase of 25%-30%) suggests that while the model effectively aligns modalities (Table 1), the Top-1 metric is bottlenecked by intra-batch semantic overlap. In many cases, multiple images in a batch are semantically consistent with a single prompt; thus, the model correctly identifies the relevant "neighborhood" in the latent space but may not assign the absolute highest similarity to the specific ground-truth index. Given our focus on retrieval utility rather than exact-match classification, we consider Top-k ($k \geq 5$) a more representative measure of the model's discriminative capabilities.

Batch Size (N)	MS-COCO			Flickr30K		
	R@1	R@5	R@10	R@1	R@5	R@10
256 (Baseline)	67.8	95.1	98.2	57.6	86.4	93.1
1024	46.8	82.0	91.4	38.5	70.8	81.3
2048	36.3	72.5	84.0	29.0	63.0	72.4
3072	30.2	64.6	80.2	23.4	56.4	69.9

Table 1: This Table shows the accuracy achieved based on batch size (gallery size) and top-k hyperparameter on two different datasets, MS-COCO and Flickr30.

To ensure that the reported metrics are not biased by the semantic composition of a single batch, we employed an iterative sub-sampling strategy. For each evaluated batch size N , we did not perform a singular inference pass. Instead, the model was evaluated over M independent iterations to achieve a cumulative evaluation pool of approximately 21,000 images ($I_{total} \approx 21k$).

For example, when testing a batch size of $N = 1024$, we conducted $M = 20$ trials. The final accuracy $\mathcal{A}$ is reported as the mean performance across all trials:

$$\mathcal{A}_{final} = \frac{1}{M} \sum_{i=1}^M \text{Recall}@K(B_i) \quad (10)$$

4.2 Model using norm loss

In our proposed multimodal framework, we initially tried to optimize semantic retrieval using a custom linear norm objective function defined as $\mathcal{L} = -z_j + \|z\|_2$, where z_j is the similarity score of the matching image-text pair and $\|z\|_2 = \sqrt{z_j^2 + \sum_{i \neq j} z_i^2}$ represents the L_2 norm of all pairwise logits in the batch. Evaluation demonstrated a severe disconnection where the training loss converged to near zero values, yet the Top-K retrieval accuracy remained equivalent to random chance. This illusion of convergence occurs because the objective evaluates the batch norm globally rather than enforcing strict ordinal ranking. Because a batch of size N contains $N - 1$ negative samples for every positive sample, the network minimizes the global norm by rapidly driving the vast majority of easy negative logits $z_i \rightarrow 0$. This shrinks $\|z\|_2$ and drastically lowers the overall loss, even if a small subset of hard negatives maintains scores where $z_{hard} > z_j$, so completely invalidating the Top-K retrieval constraints. The underlying mechanism behind this failure becomes evident when analyzing the gradient dynamics and their interplay with optimizer weight regularization. For any negative logit z_i , the partial derivative is defined as:

$$\frac{\partial \mathcal{L}}{\partial z_i} = \frac{z_i}{\|z\|_2} \quad (11)$$

This gradient dictates a downward weight update that acts with the downward regularizing pull of weight decay, creating a combined force that squashes the negative distribution to zero. Conversely, the partial derivative with respect to the positive target is defined as:

$$\frac{\partial \mathcal{L}}{\partial z_j} = -1 + \frac{z_j}{\|z\|_2} \quad (12)$$

A cancellation occurs simultaneously with the collapse of the negative distribution. As the negative logits approach zero ($\sum_{i \neq j} z_i^2 \rightarrow 0$), the batch norm simplifies entirely to the positive logit itself, such that $\|z\|_2 \rightarrow \sqrt{z_j^2} = z_j$. Substituting this limit back into the gradient equation yields a collapsed learning signal:

$$\lim_{\sum z_i^2 \rightarrow 0} \frac{\partial \mathcal{L}}{\partial z_j} = -1 + \frac{z_j}{z_j} = 0 \quad (13)$$

The exact moment the negative distribution is squashed, the upward gradient pushing the positive target completely vanishes. Then there isn't any mathematical incentive from the loss function to increase z_j , the positive logit to weight decay. So z_j is drag downward until it becomes permanently trapped at a low equilibrium point where the upward loss gradient perfectly matches the downward penalty. Because z_j is mathematically paralyzed at a low threshold, it is easily overtaken by hard negatives, preventing the network from achieving robust separation.

5 Conclusion

This project demonstrates the significant capabilities of the CLIP architecture. By scaling the model to handle batch sizes of 10,000 or more, it becomes possible to search through exceptionally large datasets efficiently. Furthermore, advanced versions like BLIP and SigLIP improve the training efficiency of this model. A planned expansion of this framework involves extending these cross-modal retrieval capabilities to the semantic audio-text domain.

References

- [1] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *arXiv preprint arXiv:2010.11929* (2020).
 - [2] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *arXiv preprint arXiv:1810.04805* (2018).
 - [3] Xiaohua Zhai et al. “LiT: Zero-Shot Transfer with Locked-image text Tuning”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 18123–18133.
 - [4] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *arXiv preprint arXiv:2103.00020* (2021).
 - [5] Tsung-Yi Lin et al. “Microsoft COCO: Common Objects in Context”. In: *European Conference on Computer Vision (ECCV)*. Springer. 2014, pp. 740–755.
- [4]. [1]. [2]. [5]. [3].

A Derivation of the Cross-Entropy Loss Formulation

In Section 2.3, the cross-entropy loss over a batch of N similarity logits $\mathbf{z} \in \mathbb{R}^N$ for a target positive index j is written as:

$$L_{\text{CE}}(\mathbf{z}, j) = -z_j + \log \left(\sum_{i=1}^N e^{z_i} \right) \quad (14)$$

Starting from the standard definition of cross-entropy for a one-hot target distribution $\mathbf{y}$ (where $y_j = 1$ and $y_i = 0$ for all $i \neq j$) and the Softmax probability p_j :

$$L_{\text{CE}}(\mathbf{y}, \mathbf{p}) = - \sum_{i=1}^N y_i \log(p_i) = -\log(p_j) \quad (15)$$

Substituting the Softmax definition $p_j = \frac{e^{z_j}}{\sum_{i=1}^N e^{z_i}}$ into the expression yields:

$$L_{\text{CE}}(\mathbf{z}, j) = -\log \left(\frac{e^{z_j}}{\sum_{i=1}^N e^{z_i}} \right) \quad (16)$$

Applying the logarithmic identity $\log(\frac{A}{B}) = \log(A) - \log(B)$:

$$L_{\text{CE}}(\mathbf{z}, j) = - \left[\log(e^{z_j}) - \log \left(\sum_{i=1}^N e^{z_i} \right) \right] \quad (17)$$

Distributing the negative sign and simplifying using the inverse property $\log(e^{z_j}) = z_j$:

$$L_{\text{CE}}(\mathbf{z}, j) = -z_j + \log \left(\sum_{i=1}^N e^{z_i} \right) \quad (18)$$